

# Test Plan — Mobile Trading App (iOS & Android)

Context: fintech startup, first public release in two weeks, no existing test suite or QA documentation, real user funds involved. This plan is scoped to be executable in that window, not a theoretical ideal.

## 1. Objective

Verify the app is safe and correct to put in front of the public with real funds — prioritizing money-correctness, account security, and release blockers over comprehensive feature coverage, given the timeline.

## 2. Scope

### In scope

- Onboarding & KYC flow (as far as it can be tested without submitting real personal/financial documents into a live pipeline)
- Auth: sign up, log in, biometric login, session expiry, logout
- Funding: deposit flow (sandbox/test rails)
- Core trading: viewing markets, placing/cancelling orders, order types supported at launch, balance and portfolio views
- Withdrawals (sandbox/test rails)
- Notifications tied to trading activity (order filled, price alerts)
- Navigation & layout across both platforms
- App Store / Play Store submission requirements for financial apps

### Out of scope for this cycle (flagged, not ignored)

- Full localization matrix beyond the launch-market language(s)
- Non-critical settings/profile screens with no financial impact
- Load/stress testing at a scale beyond expected launch traffic
- Full accessibility audit (basic checks only in this cycle; scheduled as a fast-follow)

## 3. Test levels

| Level | Owner | Runs         | Purpose                                  |
|-------|-------|--------------|------------------------------------------|
| Unit  | Dev   | Every commit | Business logic, calculations, validation |

| Level                                        | Owner    | Runs                             | Purpose                                                                                                       |
|----------------------------------------------|----------|----------------------------------|---------------------------------------------------------------------------------------------------------------|
| API / contract                               | QA + Dev | Every PR, nightly                | Auth, orders, balances, deposits/withdrawals — the layer most likely to catch money-correctness bugs, fastest |
| E2E smoke ( @smoke )                         | QA       | Every merge to release branch    | Critical path only, both platforms, ~10-15 min                                                                |
| E2E regression ( @regression )               | QA       | Nightly, every release candidate | Broader functional + negative coverage, real device matrix                                                    |
| Manual exploratory                           | QA       | Every sprint, every RC           | Unscripted, targeted at what's new/risky                                                                      |
| Non-functional (perf/security/accessibility) | QA       | Scheduled, pre-release           | Not run on every PR — too slow — but mandatory before RC sign-off                                             |

## 4. Test approach by area

---

**Functional — money paths (highest priority)** Order placement/cancellation, balance updates, deposit/withdrawal, fee and rounding calculations. Tested at API level first (fast, deterministic), then a thin E2E layer confirming the UI reflects the same truth. Includes boundary values (minimum trade size, max decimal precision) and concurrency cases (rapid double-submit, cancel racing a fill).

**Functional — navigation & layout** Top navigation renders and routes correctly; standard desktop/tablet/phone breakpoints render without overflow or clipped controls; deep links open the correct screen.

**Functional — content** Marketing content renders in the correct region; outbound links (app stores, support, legal) resolve; static content pages (About/Why MultiBank-equivalent) render expected headings and copy.

**Security** Session expiry is enforced and communicated to the user (no silent "logged in but every write fails" state); biometric login has a working fallback; no sensitive data (tokens, balances) in plaintext logs or unencrypted local storage; certificate pinning active in release builds.

**Performance / reliability** App behavior under degraded network (3G throttle, mid-request drop) — specifically whether an in-flight order's state is ever ambiguous to the user. Cold start time. Behavior when backgrounded mid-transaction and resumed.

**Compliance** App Store / Play Store guidelines for financial apps (required disclosures, licensing text, age rating) verified against a checklist early in the cycle — not discovered at submission.

**Device / OS matrix** Prioritized by real usage data if available; otherwise top 2 supported iOS versions and the most common Android OEM/OS combinations, run on a cloud device farm for the long tail rather than physical-device-only coverage.

## 5. Entry / exit criteria

---

**Entry** (starting a test cycle on a build): build installs on all matrix devices, smoke suite passes, no P1 open from the previous cycle.

**Exit** (release candidate approved): full regression suite green (or every red explicitly triaged and waived with a written reason by the QA lead), no open P1/P2, App Store/Play Store compliance checklist complete, rollback/kill-switch plan confirmed working.

## 6. Tools (indicative — adapt to what the team already has in place)

---

- E2E: Playwright (web surfaces) / Appium or platform-native (Detox for RN, XCUITest/Espresso for native) for the mobile app itself
- API: Postman/Newman or the same Playwright `request` context style used in this repo
- Device matrix: BrowserStack App Live / Firebase Test Lab
- Crash/monitoring: Crashlytics or Sentry, wired to alert QA + dev on spike
- CI: GitHub Actions — unit + API + smoke on every PR, full regression nightly and pre-release

## 7. Two-week schedule (compressed, risk-first)

---

| Days  | Focus                                                                                                    |
|-------|----------------------------------------------------------------------------------------------------------|
| 1–2   | Exploratory pass on both platforms, risk mapping, environment/access setup, triage process established   |
| 3–5   | API/contract test coverage on all money-touching endpoints; smoke E2E for the critical path              |
| 6–9   | Broaden regression coverage; security/session checks; device matrix runs; App Store compliance checklist |
| 10–11 | Bug fixing + retest cycle; performance/degraded-network pass                                             |
| 12    | Full-team bug bash                                                                                       |
| 13    | Release candidate cut, full regression + compliance sign-off                                             |
| 14    | Go/no-go review, submission                                                                              |

## 8. Assumptions

---

- Sandboxed payment/KYC rails are available for testing (no real funds moved during test execution).
- The team can provide API documentation or a Postman collection, or QA can reverse-engineer it from network traffic early in week one.
- A staging environment with production-equivalent behavior exists or can be stood up in the first two days.